



2η Εργαστηριακή Άσκηση

Ψηφιακά Συστήματα VLSI

Παναγιώτης Γερασιμόπουλος 03115208
personal@devcol.com

Contents

1. Full Adder	1
1.1. Implementation	1
1.2. Test Bench	2
2. Pipelined 4 Bit Adder	3
2.1. Implementation	4
2.2. Test Bench	6
3. Array Multiplier	7
3.1. Implementation	8
3.2. Test Bench	12

1. Full Adder

Το κύκλωμα είναι υλοποιείται με τις συναρτήσεις

$$\begin{aligned} Sum &= A \oplus B \oplus C_{in} \\ C_{out} &= AB + C_{in}A + C_{in}B \end{aligned} \quad (1.1)$$

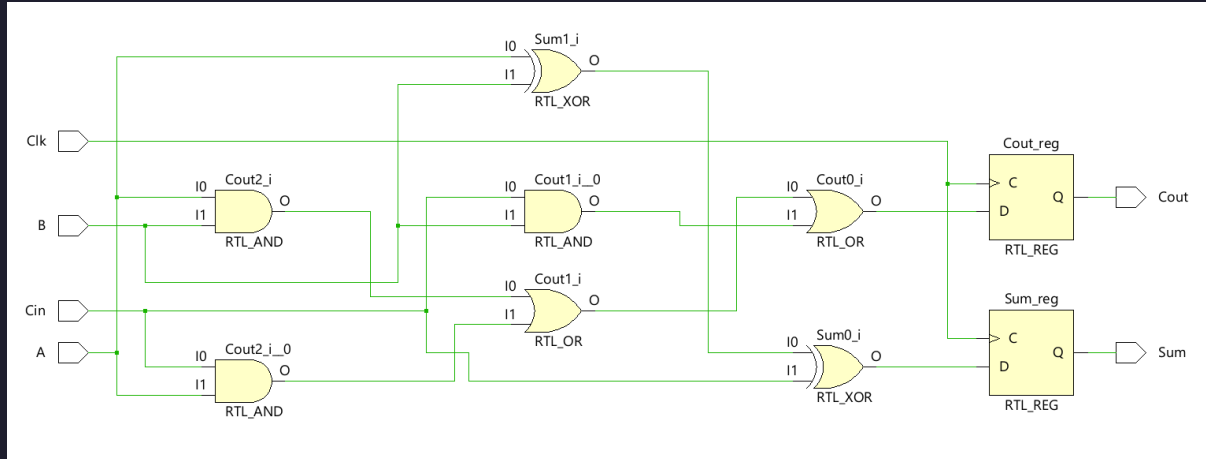


Figure 2: RTL

1.1. Implementation

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity FA is
6   Port (
7     Clk, A, B, Cin : in std_logic;
8     Sum : out std_logic;
9     Cout : out std_logic
10  );
11 end FA;
12
13 architecture Behavioral of FA is
14 begin
15
16   process(Clk) begin
17     if rising_edge(Clk) then
18       Sum <= A xor B xor Cin;
19       Cout <= (A and B) or (Cin and A) or (Cin and B);
20     end if;
21   end process;
22 end architecture;
23

```

Το οποίο μας δίνει critical path 4.076ns.

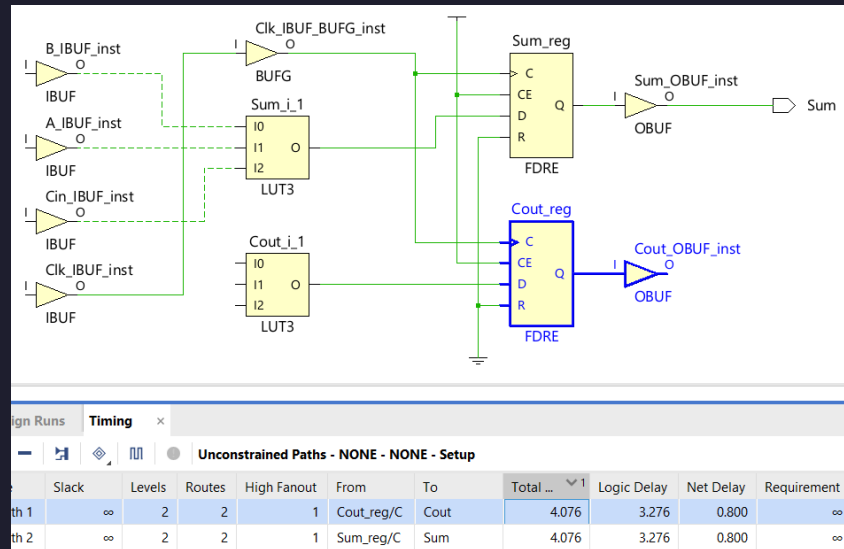


Figure 3: Critical path 4.076

1.2. Test Bench

Το κύκλωμα είναι αρκετά απλό ώστε να μπορούμε να το ελέγξουμε για όλες τις τιμές αληθείας.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4
5 entity tb_FA is
6 end tb_FA;
7
8 architecture Behavioral of tb_FA is
9
10     component FA
11     Port (
12         A, B, Cin, Clk : in std_logic;
13         Sum : out std_logic;
14         Cout : out std_logic
15     );
16 end component;
17
18     signal Clk : std_logic := '0';
19     signal A : std_logic := '0';
20     signal B : std_logic := '0';
21     signal Cin : std_logic := '0';
22     signal Sum : std_logic;
23     signal Cout : std_logic;
24
25     constant clk_period : time := 20 ns;
26
27 begin
28     uut: FA PORT MAP (Clk, A, B, Cin, Sum, Cout);
29
30     -- Clock inf loop
31     process begin
32         clk <= '0';
33         wait for clk_period/2;
34         clk <= '1';
35         wait for clk_period/2;
36     end process;
37
38     process begin

```

```

39   A <= '0'; B <= '0'; Cin <= '0';
40   wait for clk_period;
41   A <= '0'; B <= '0'; Cin <= '1';
42   wait for clk_period;
43   A <= '0'; B <= '1'; Cin <= '0';
44   wait for clk_period;
45   A <= '0'; B <= '1'; Cin <= '1';
46   wait for clk_period;
47   A <= '1'; B <= '0'; Cin <= '0';
48   wait for clk_period;
49   A <= '1'; B <= '0'; Cin <= '1';
50   wait for clk_period;
51   A <= '1'; B <= '1'; Cin <= '0';
52   wait for clk_period;
53   A <= '1'; B <= '1'; Cin <= '1';
54   wait for clk_period;
55   wait;
56   end process;
57   end architecture;
58

```



Figure 4: Test Bench

2. Pipelined 4 Bit Adder

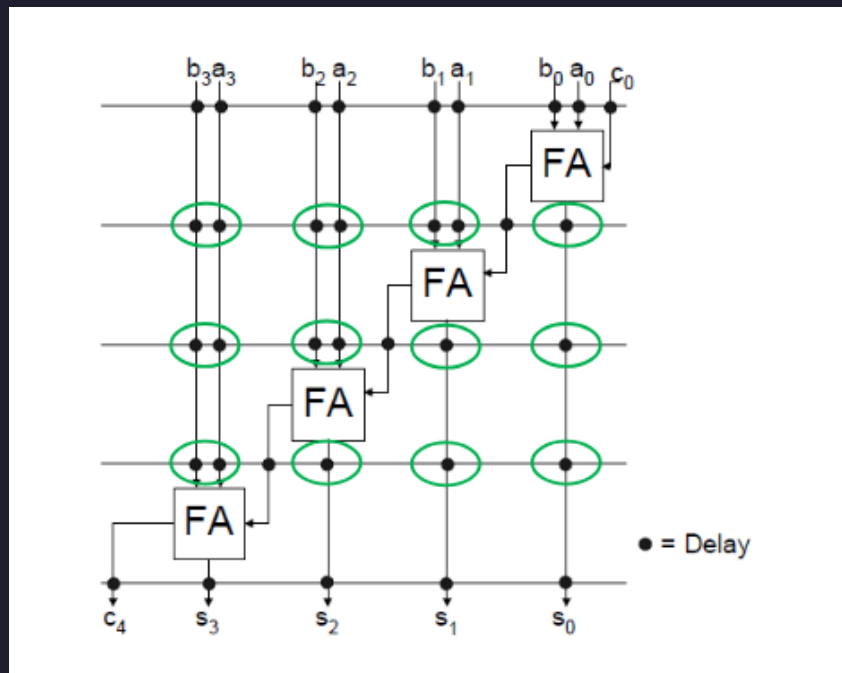


Figure 5: Θέσεις επιπλέον FlipFlop

Ο αθροιστής λειτουργεί συνδέοντας 4 Full Adder του προηγούμενου ερωτήματος όπου σε κάθε κύκλο ρολογιού το C_{out} του ενός προωθείτε στο C_{in} του επόμενου σταδίου. Επίσης προσθέτουμε αρκετή καθυστέρηση πριν την είσοδο του κάθε Full Adder ώστε το C_{in} που

προέρχεται από τον προηγούμενο να έχει προλάβει αν υπολογιστεί.

Σε σχέση με τον κασκοδικό της ασκήσεις 1, όπου όλο το το άθροισμα υπολογιζόταν σε ένα κύκλο, εδώ υπολογίζουμε το κάθε άθροισμα σε πολλούς κύκλους με όμως λιγότερη λογική μεταξύ κύκλων και το pipeline μας επιτρέπει να έχουμε in flight πολλά αθροίσματα. Αποτέλεσμα όλου αυτού είναι ένα κύκλωμα με delay 4 κύκλους, throughput 1 και μικρότερο delay στο critical path από τον κασκοδικό, 4.076ns αντί για 5.970ns, με κόστος όμως περισσότερο υλικό, κυρίως flipflop.

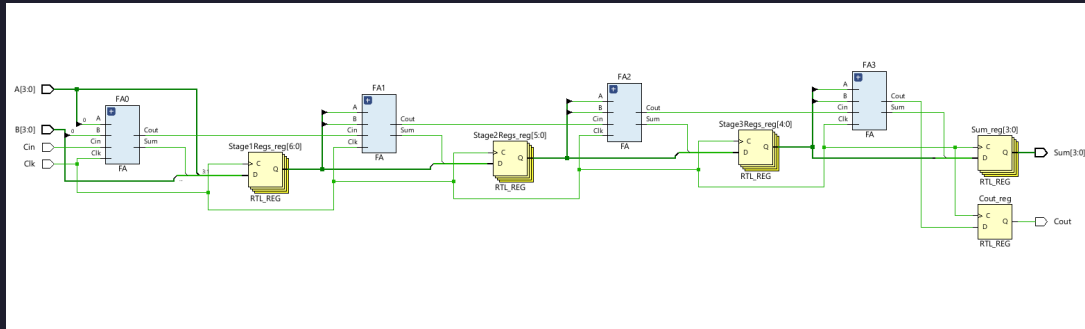


Figure 6: RTL

2.1. Implementation

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity FA4Transmission is
6  Port(
7      Clk, Cin : in std_logic;
8      A, B : in std_logic_vector(3 downto 0);
9      Sum : out std_logic_vector(3 downto 0);
10     Cout : out std_logic
11 );
12 end FA4Transmission;
13
14 architecture Structural of FA4Transmission is
15     component FA
16     Port(
17         Clk, A, B, Cin : in std_logic;
18         Sum, Cout : out std_logic
19     );
20 end component;
21
22     signal Stage1Regs : std_logic_vector(6 downto 0);
23     signal Stage2Regs : std_logic_vector(5 downto 0);
24     signal Stage3Regs : std_logic_vector(4 downto 0);
25
26     signal C : std_logic_vector(3 downto 0);
27     signal SumWires : std_logic_vector(3 downto 0);
28 begin
29
30     FA0: FA port map (Clk, A(0), B(0), Cin, SumWires(0), C(0));
31     FA1: FA port map (Clk, Stage1Regs(1), Stage1Regs(4), C(0), SumWires(1), C(1));
32     FA2: FA port map (Clk, Stage2Regs(2), Stage2Regs(4), C(1), SumWires(2), C(2));
33     FA3: FA port map (Clk, Stage3Regs(3), Stage3Regs(4), C(2), SumWires(3), C(3));
34
35     process(Clk) begin
36         if rising_edge(Clk) then
37
38             Stage1Regs(3 downto 1) <= A(3 downto 1);
39             Stage1Regs(6 downto 4) <= B(3 downto 1);

```

```

40     Stage1Regs(0) <= SumWires(0);
41
42     Stage2Regs(0) <= Stage1Regs(0);
43     Stage2Regs(3 downto 2) <= Stage1Regs(3 downto 2);
44     Stage2Regs(5 downto 4) <= Stage1Regs(6 downto 5);
45     Stage2Regs(1) <= SumWires(1);
46
47     Stage3Regs(1 downto 0) <= Stage2Regs(1 downto 0);
48     Stage3Regs(3) <= Stage2Regs(3);
49     Stage3Regs(4) <= Stage2Regs(5);
50     Stage3Regs(2) <= SumWires(2);
51
52     Sum(2 downto 0) <= Stage3Regs(2 downto 0);
53     Sum(3) <= SumWires(3);
54     Cout <= C(3);
55
56 end if;
57 end process;
58 end architecture;
59

```

Το οποίο μας δίνει critical path 4.076ns

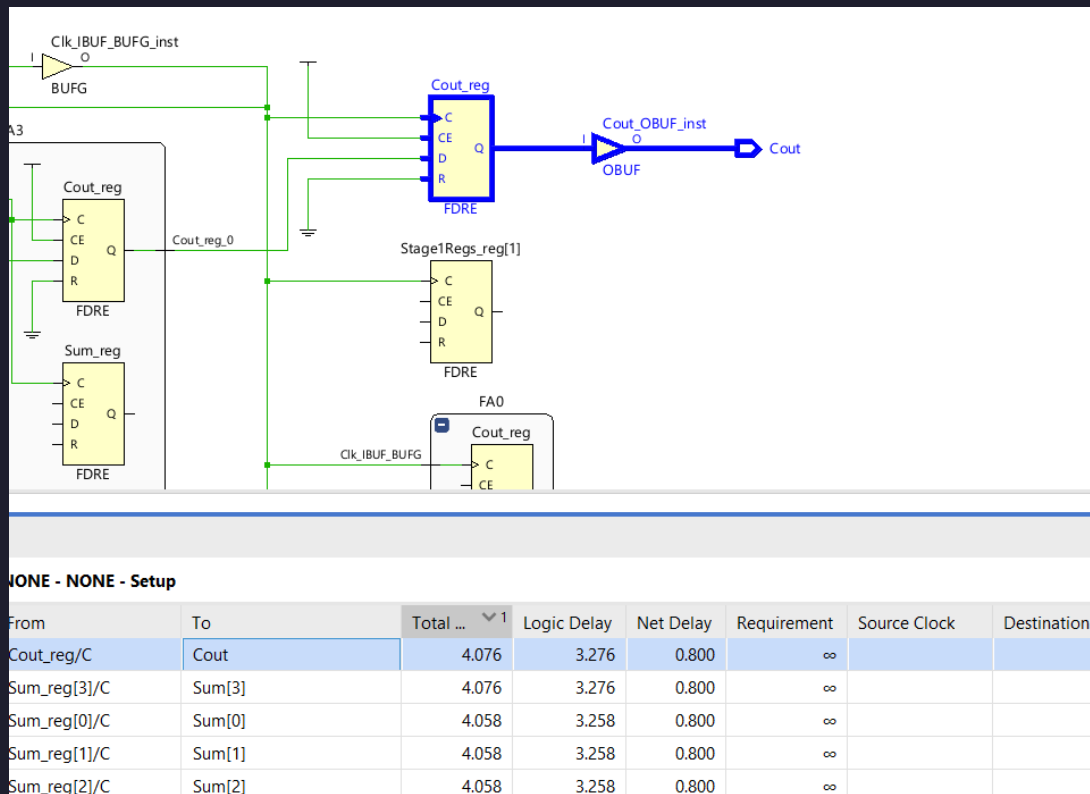


Figure 7: Critical path 4.076

2.2. Test Bench

Θα κάνουμε probe συγκεκριμένες τιμές οπου στοχεύουν σε διαφορετικές συμπεριφορές του κυκλώματος, πχ με C_{in} , C_{out} η χωρίς, και τα αποτελέσματα που αναμένουμε αναγράφονται στα σχόλια του κώδικα.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity tb_FA4Transmission is
6  end tb_FA4Transmission;
7
8  architecture behavior of tb_FA4Transmission is
9    component FA4Transmission
10      Port(
11        Clk, Cin : in std_logic;
12        A, B      : in std_logic_vector(3 downto 0);
13        Sum       : out std_logic_vector(3 downto 0);
14        Cout      : out std_logic
15      );
16    end component;
17
18    signal Clk : std_logic := '0';
19    signal Cin : std_logic := '0';
20    signal A   : std_logic_vector(3 downto 0) := (others => '0');
21    signal B   : std_logic_vector(3 downto 0) := (others => '0');
22    signal Sum : std_logic_vector(3 downto 0);
23    signal Cout : std_logic;
24
25    constant clk_period : time := 10 ns;
26  begin
27
28    uut: FA4Transmission port map (
29      Clk => Clk,
30      Cin => Cin,
31      A   => A,
32      B   => B,
33      Sum => Sum,
34      Cout => Cout
35    );
36
37    process begin
38      Clk <= '0';
39      wait for clk_period/2;
40      Clk <= '1';
41      wait for clk_period/2;
42    end process;
43
44    process begin
45      wait for clk_period;
46      -- Sum = 0101 Cout = 0
47      A <= "0011"; B <= "0010"; Cin <= '0';
48      wait for clk_period;
49      -- Sum = 1110 Cout = 0
50      A <= "0111"; B <= "0111"; Cin <= '0';
51      wait for clk_period;
52      -- Sum = 0000 Cout = 1
53      A <= "1111"; B <= "0001"; Cin <= '0';
54      wait for clk_period;
55      -- Sum = 1111 Cout = 1
56      A <= "1111"; B <= "1111"; Cin <= '1';
57      wait for clk_period;
58      -- flush pipeline
59      A <= "0000"; B <= "0000"; Cin <= '0';
60      wait for clk_period * 5;
61

```

```

62     wait;
63   end process;
64 end architecture;
65

```

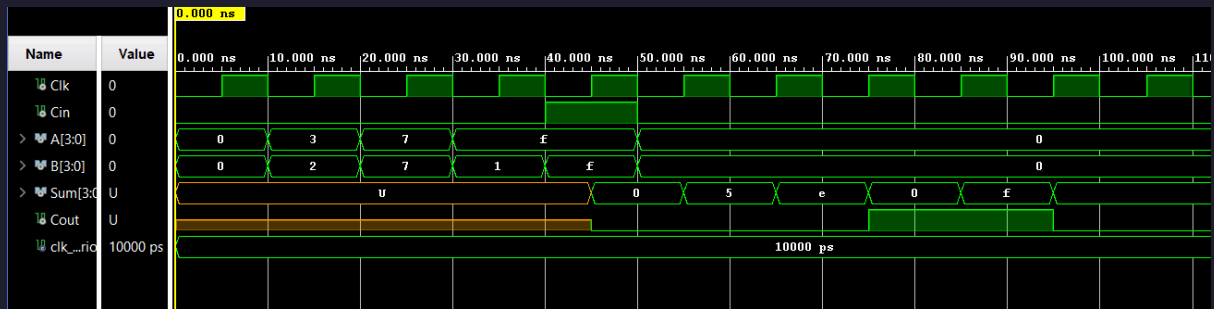
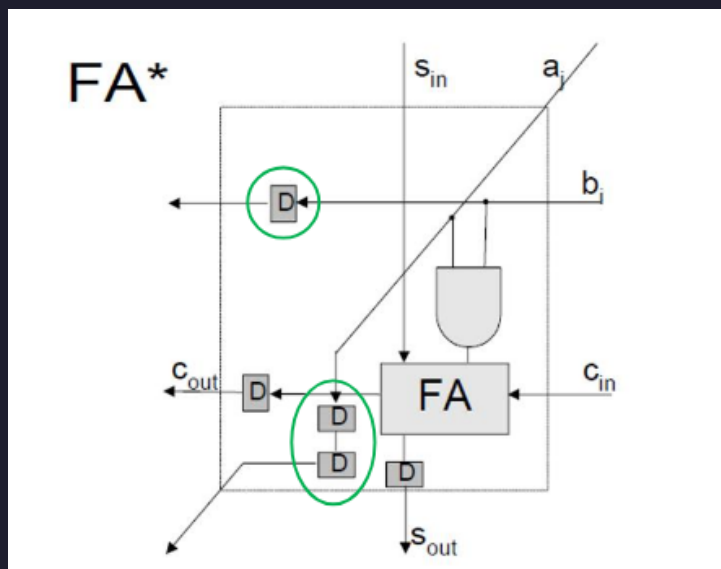
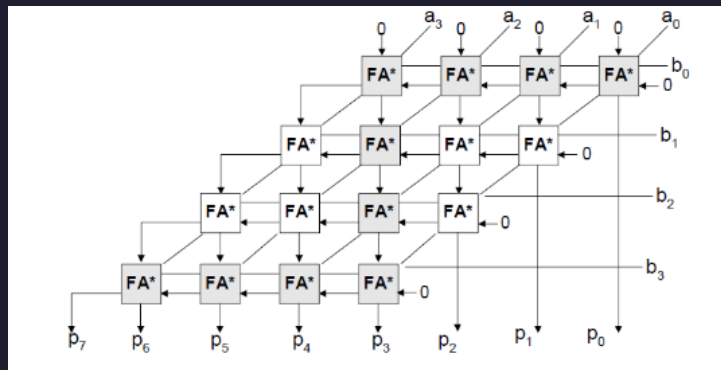


Figure 8: Test Bench

3. Array Multiplier



Παραπάνω φαίνονται το ολοκληρωμένο κύκλωμα καθώς και το βασικό δομικό στοιχείο που θα χρησιμοποιήσουμε. Η διάδοση του υπολογισμού γίνεται σε μορφή κύματος αρχίζοντας από το a_0, b_0 . Αποτέλεσμα είναι ότι πρέπει να προσθέσουμε καθυστέρηση στην είσοδο του κάθε αθροιστή και στην έξοδο του συνολικού για να δρομολογηθούν στο output όλα τα σχετικά bit ταυτόχρονα. Ποιο συγκεκριμένα χρειαζόμαστε καθυστέρηση 1 κύκλο

μεταξύ κάθε a, 2 μεταξύ κάθε b και το product αρχίζει με καθυστέρηση 9 μέχρι 0 με βήμα 2 μέχρι το 4 και μετά βήμα 1 μέχρι το 0.

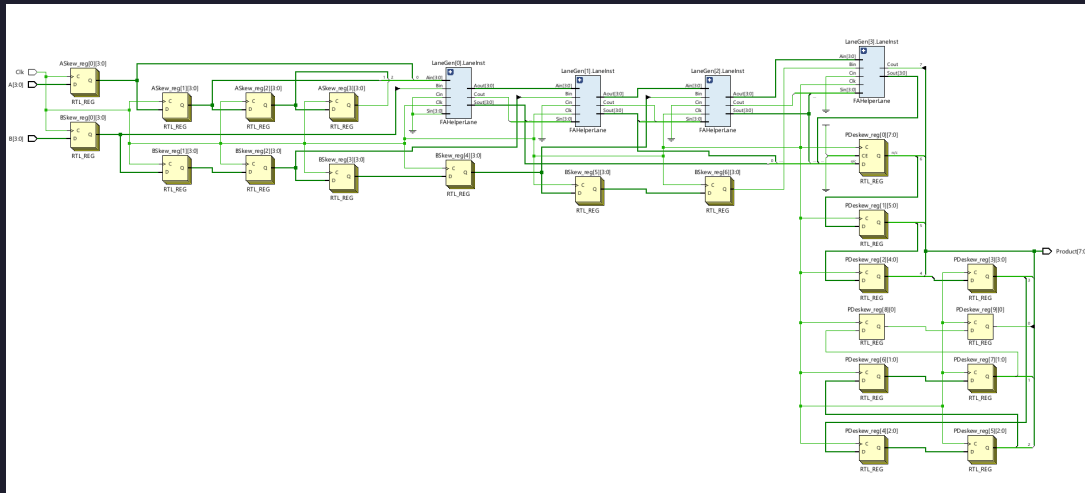


Figure 11: RTL

3.1. Implementation

Στις γραμμές 1-48 υλοποιούμε το βασικό δομικό στοιχείο που παρουσιάσαμε παραπάνω και στις γραμμές 50-96 δημιουργούμε στάδια από τα FA* με προσοχή να προσθέσουμε καθυστέρηση στο C_{out} .

Γραμμές 100-193 είναι το implementation του multiplier όπου συνδέουμε 4 από τα παραπάνω στάδια. Το μονό ίσως λεπτό σημείο είναι ότι αντί για να προσθέσουμε την κάθε καθυστέρηση με το χέρι, βάζουμε πλέγματα από register όπου το καθένα τροφοδοτεί το επόμενο. Έτσι μπορούμε να ενώσουμε το input που θέλουμε στην αρχή του πλέγματος και άπλα να το ξαναπάρουμε σε όποιο στάδιο μας βολεύει. Για παράδειγμα στις γραμμές 144-150 προσθέτουμε στην αρχή των αντίστοιχων πλεγμάτων καθυστέρησης το A, B και μετά με ένα for loop προωθούμε το κάθε στάδιο του πλέγματος στο επόμενο, τέλος στην γραμμή 169 παίρνουμε από το πλέγμα σήμα προς την είσοδο του πρώτου σταδίου μετά από κατάλληλη καθυστέρηση.

```

1  -- NOTE(acol): Helpers
2  -----
3  library IEEE;
4  use IEEE.STD_LOGIC_1164.ALL;
5
6  entity FAHelper is
7      Port(
8          Clk, Ain, Bin : in std_logic;
9          Sin           : in  std_logic;
10         Cin           : in  std_logic;
11         Sout          : out std_logic;
12         Cout          : out std_logic;
13         Aout, Bout    : out std_logic
14     );
15 end FAHelper;
16
17 architecture Structural of FAHelper is
18     component FA
19         Port(
20             Clk : in  std_logic;
21             A   : in  std_logic;

```

```

22     B    : in  std_logic;
23     Cin  : in  std_logic;
24     Sum  : out std_logic;
25     Cout : out std_logic
26 );
27 end component;
28
29 signal LocalProduct : std_logic;
30 signal ADelay : std_logic_vector(1 downto 0);
31 signal BDelay : std_logic;
32 begin
33     LocalProduct <= Ain and Bin;
34     FA0: FA port map(Clk, LocalProduct, Sin, Cin, Sout, Cout);
35
36     process(Clk) begin
37         if rising_edge(Clk) then
38             ADelay(0) <= Ain;
39             ADelay(1) <= ADelay(0);
40
41             BDelay <= Bin;
42         end if;
43     end process;
44
45     Aout <= ADelay(1);
46     Bout <= BDelay;
47
48 end Structural;
49
50 -- Helper lane
51 library IEEE;
52 use IEEE.STD_LOGIC_1164.ALL;
53
54 entity FAHelperLane is
55     Port(
56         Clk, Bin, Cin : in std_logic;
57         Ain, Sin      : in std_logic_vector(3 downto 0);
58         Aout, Sout    : out std_logic_vector(3 downto 0);
59         Cout          : out std_logic
60     );
61 end FAHelperLane;
62
63 architecture Structural of FAHelperLane is
64     component FAHelper
65         Port(
66             Clk, Ain, Bin : in std_logic;
67             Sin          : in std_logic;
68             Cin          : in std_logic;
69             Sout         : out std_logic;
70             Cout         : out std_logic;
71             Aout, Bout   : out std_logic
72         );
73     end component;
74
75     -- B(4) is dead code eliminated
76     signal C, B : std_logic_vector(4 downto 0) := (others => '0');
77
78 begin
79     B(0) <= Bin;
80     C(0) <= Cin;
81
82     LaneGen: for i in 0 to 3 generate
83         Cell: FAHelper port map (
84             Clk, Ain(i), B(i), Sin(i), C(i), Sout(i), C(i + 1), Aout(i), B(i + 1)
85         );
86     end generate LaneGen;
87
88     -- stagger last output
89     process(Clk) begin
90         if rising_edge(Clk) then
91             Cout <= C(4);

```

```

92     end if;
93   end process;
94 end architecture;
95
96
97 -----
98
99 -- NOTE(acol): Multiplier implementation
100 library IEEE;
101 use IEEE.STD_LOGIC_1164.ALL;
102
103 entity SystolicMultiplier4Bit is
104   Port(
105     Clk      : in  std_logic;
106     A        : in  std_logic_vector(3 downto 0);
107     B        : in  std_logic_vector(3 downto 0);
108     Product  : out std_logic_vector(7 downto 0)
109   );
110 end SystolicMultiplier4Bit;
111
112 architecture Structural of SystolicMultiplier4Bit is
113
114   component FAHelperLane
115     Port(
116       Clk : in  std_logic;
117       Bin : in  std_logic;
118       Cin : in  std_logic;
119       Ain : in  std_logic_vector(3 downto 0);
120       Sin : in  std_logic_vector(3 downto 0);
121       Aout : out std_logic_vector(3 downto 0);
122       Sout : out std_logic_vector(3 downto 0);
123       Cout : out std_logic
124     );
125   end component;
126
127   type VectorArray is array (0 to 4) of std_logic_vector(3 downto 0);
128   type BitArray    is array (0 to 4) of std_logic;
129   type Array4Bit    is array (0 to 6) of std_logic_vector(3 downto 0);
130   type Array8Bit    is array (0 to 9) of std_logic_vector(7 downto 0);
131
132   signal RowAin, RowAout, RowSin, RowSout : VectorArray := (others => (others => '0'));
133   signal RowCout                          : BitArray    := (others => '0');
134
135   signal ASkew  : Array4Bit := (others => "0000");
136   signal BSkew  : Array4Bit := (others => "0000");
137   signal PDeskew : Array8Bit := (others => "00000000");
138
139 begin
140   process(Clk) begin
141     -- Stagger grids
142     if rising_edge(Clk) then
143       -- A B skew grid inputs
144       ASkew(0) <= A;
145       BSkew(0) <= B;
146       -- A B skew grid progression
147       for i in 0 to 5 loop
148         ASkew(i+1) <= ASkew(i);
149         BSkew(i+1) <= BSkew(i);
150       end loop;
151
152       -- Product deskew grid inputs
153       PDeskew(0)(0) <= RowSout(0)(0);
154       PDeskew(0)(1) <= RowSout(1)(0);
155       PDeskew(0)(2) <= RowSout(2)(0);
156       PDeskew(0)(3) <= RowSout(3)(0);
157       PDeskew(0)(4) <= RowSout(3)(1);
158       PDeskew(0)(5) <= RowSout(3)(2);
159       PDeskew(0)(6) <= RowSout(3)(3);
160       -- Product deskew grid progression
161       for i in 0 to 8 loop

```

```

162     PDeskew(i+1) <= PDeskew(i);
163   end loop;
164
165   end if;
166 end process;
167
168 -- Row 1 inputs
169 RowAin(0) <= ASkew(3)(3) & ASkew(2)(2) & ASkew(1)(1) & ASkew(0)(0);
170 RowSin(0) <= "0000";
171
172 -- Row gen and connections
173 LaneGen: for i in 0 to 3 generate
174   LaneInst: FAHelperLane port map (
175     Clk, BSkew(i * 2)(i), '0', RowAin(i), RowSin(i), RowAout(i), RowSout(i), RowCout(i)
176   );
177
178   RowAin(i+1) <= RowAout(i);
179   RowSin(i+1)(2 downto 0) <= RowSout(i)(3 downto 1);
180   RowSin(i+1)(3) <= RowCout(i);
181 end generate LaneGen;
182
183 -- Outputs
184 Product(0) <= PDeskew(9)(0);
185 Product(1) <= PDeskew(7)(1);
186 Product(2) <= PDeskew(5)(2);
187 Product(3) <= PDeskew(3)(3);
188 Product(4) <= PDeskew(2)(4);
189 Product(5) <= PDeskew(1)(5);
190 Product(6) <= PDeskew(0)(6);
191 Product(7) <= RowCout(3);
192 end Structural;
193

```

Το οποίο µα δίνει critical path 4.076ns

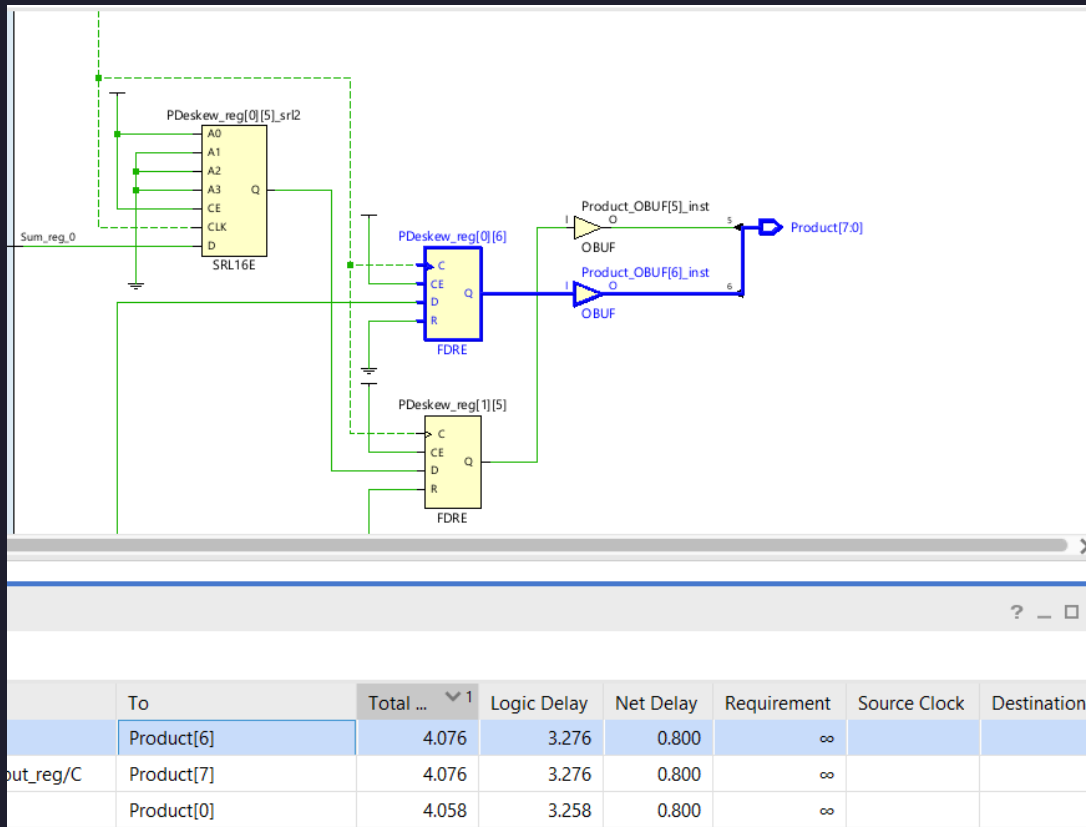


Figure 12: Critical path 4.076

3.2. Test Bench

Όπως παραπάνω θα κάνουμε probe συγκεκριμένες τιμές, και οι αναμενόμενες απαντήσεις είναι γραμμένες σε comment στον κώδικα.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity TbSystolicMultiplier4Bit is
6  end TbSystolicMultiplier4Bit;
7
8  architecture Behavior of TbSystolicMultiplier4Bit is
9      component SystolicMultiplier4Bit
10         Port(
11             Clk      : in  std_logic;
12             A        : in  std_logic_vector(3 downto 0);
13             B        : in  std_logic_vector(3 downto 0);
14             Product   : out std_logic_vector(7 downto 0)
15         );
16     end component;
17
18     signal Clk      : std_logic := '0';
19     signal A        : std_logic_vector(3 downto 0) := (others => '0');
20     signal B        : std_logic_vector(3 downto 0) := (others => '0');
21     signal Product  : std_logic_vector(7 downto 0);
22
23     constant ClkPeriod : time := 10 ns;
24     begin
25         UUT: SystolicMultiplier4Bit port map (
26             Clk      => Clk,
27             A        => A,
28             B        => B,
29             Product  => Product
30         );
31
32         process begin
33             Clk <= '0';
34             wait for ClkPeriod / 2;
35             Clk <= '1';
36             wait for ClkPeriod / 2;
37         end process;
38
39         process begin
40             wait for ClkPeriod * 2;
41             -- 2 x 3 = 6
42             A <= "0010"; B <= "0011";
43             wait for ClkPeriod;
44             -- 4 x 4 = 16
45             A <= "0100"; B <= "0100";
46             wait for ClkPeriod;
47             -- 15 x 1 = 15
48             A <= "1111"; B <= "0001";
49             wait for ClkPeriod;
50             -- 10 x 2 = 20
51             A <= "1010"; B <= "0010";
52             wait for ClkPeriod;
53             -- 15 x 15 = 225
54             A <= "1111"; B <= "1111";
55             wait for ClkPeriod;
56             -- 1 x 1 = 1
57             A <= "0001"; B <= "0001";
58             wait for ClkPeriod;
59             -- flush
60             A <= "0000"; B <= "0000";
61             wait for ClkPeriod * 10;
62
63             wait;

```

```
64   end process;  
65   end Behavior;  
66
```

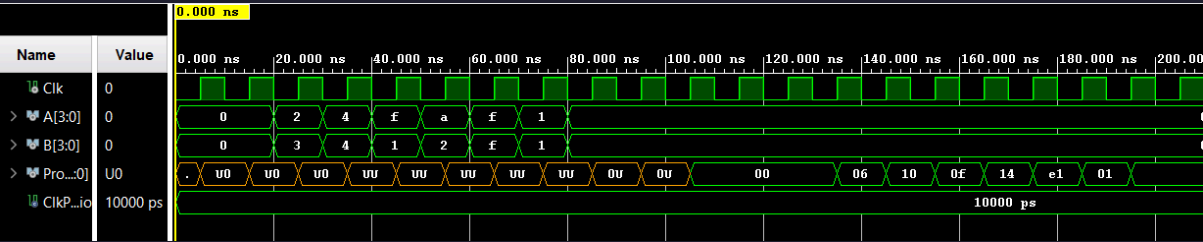


Figure 13: Test Bench